

Comparative Analysis of Control Strategies for 6-DOF Robotic Arms

Abhijit Kaluri kaluri.a@northeastern.edu
Harshil Mistry mistry.har@northeastern.edu
Kaustubh Chaudhari chaudhari.kau@northeastern.edu
Ronak Bhanushali bhanushali.r@husky.neu.edu

December 2024

1 Problem Identification and Scope

The problem we aim to address is the development of effective control strategies for a robotic arm with six degrees of freedom (6-DOF). This type of robotic arm needs to accurately perform tasks requiring complex movements, such as reaching specified locations or following intricate trajectories. The challenge lies in ensuring that the robotic arm can receive desired input positions and calculate the appropriate torque inputs for its joints to achieve these movements.

The scope of this project is to design and implement various controllers to facilitate precise control of the robotic arm. These controllers are tasked with translating high-level movement commands into low-level torque signals for each joint, ensuring the arm follows the desired trajectory efficiently and accurately. The objective is to compare the effectiveness of these controllers in terms of performance metrics such as trajectory tracking accuracy, control stability, and adaptability.

2 Background and Motivation

Robotic arms with six degrees of freedom (6-DOF) are widely used in industries such as automotive, electronics, and healthcare, enabling tasks like assembly, welding, and surgery. Their ability to automate complex tasks enhances productivity and reduces operational costs. However, controlling these robots is challenging due to their high complexity and multiple degrees of freedom.

Traditional control methods, such as Proportional Integral Derivative (PID) controllers, are simple and effective for basic applications. They minimize the error between the desired and actual trajectory, making them suitable for systems with fewer degrees of freedom. However, PID controllers struggle with the coupled dynamics and nonlinearities present in 6-DOF systems, leading to limited performance in more complex tasks.

Linear Quadratic Regulators (LQR) offer a more sophisticated approach by minimizing a cost function that balances tracking accuracy and control effort. LQR is particularly effective for systems with multiple degrees of freedom, as it handles coupled dynamics well. However, it requires accurate system modeling and tuning to achieve optimal performance.

The Computed Torque Controller (CTC) uses the robot's dynamic model, accounting for inertia, Coriolis forces, and gravity, to compute precise torque commands for each joint. This makes CTC well-suited for high-precision trajectory tracking. However, it can be computationally expensive and sensitive to model inaccuracies.

This project compares PID and LQR, and explores CTC to determine which control strategy offers the best performance for a 6-DOF robotic arm in terms of trajectory tracking and robustness.

3 State of the Art and Main Contributions

The control of robotic arms with six degrees of freedom (6-DOF) is an area of extensive research, with various techniques proposed and implemented to address the challenges of precise motion control. Traditional methods such as Proportional Integral Derivative (PID) controllers are straightforward and widely used due to their simplicity and effectiveness for Single-Input-Single-Output (SISO) systems. However, PID controllers often fall short in handling dynamic interactions between joints for complex Multi-Input-Multi-Output (MIMO) systems like a 6-DOF robotic arm.

Alternative approaches such as Computed Torque Control (CTC), Linear Quadratic Regulator (LQR), and Rapidly-exploring Random Tree (RRT) motion planning algorithms have demonstrated improvements in handling the complexities of robotic arm control. Additionally, advancements in Machine Learning (ML) techniques have enabled data-driven controllers that can learn from experience and adapt to complex environments. These methods aim to enhance trajectory tracking, reduce control errors, and improve overall efficiency.

The main contribution of this project lies in comparing the performance of these control strategies for a 6-DOF robotic arm in a simulated environment. By implementing and evaluating these controllers, we aim to provide insights into their respective strengths, limitations, and potential use cases.

4 6-Axis Robot Model

In this project, a single model of the 6-axis robotic arm was developed in MATLAB/Simulink. MATLAB was chosen as the primary environment for implementing and testing various control strategies, including Proportional Integral Derivative (PID), Computed Torque Control (CTC), and Linear Quadratic Regulator (LQR) controllers.

The MATLAB/Simulink model provides a representation of the 6-axis robotic

arm, accurately capturing its kinematics and dynamics. The model enables the simulation of joint movements and torque inputs required to follow desired trajectories. Controllers were designed and implemented in Simulink, leveraging its robust toolbox for real-time simulation and performance analysis. The model was tested on predefined trajectories, allowing for a thorough evaluation of the controllers' performance in tracking accuracy and stability.

Below is a depiction of the 6-axis robot as rendered in Simulink, showing its structural configuration and the simulated dynamics environment.

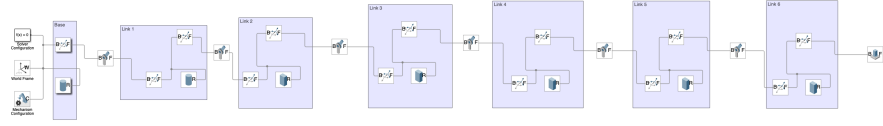


Figure 1: Simscape model of 6 Axis Robotic arm

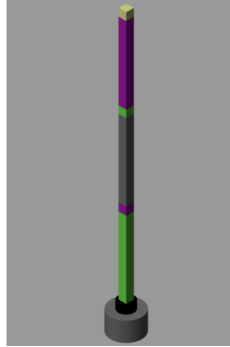


Figure 2: Simscape render of Simplified model

5 Controller Implementation

5.1 PID Controller

A Proportional Integral Derivative (PID) controller was implemented as a baseline control strategy for the 6-axis robotic arm. While PID controllers are straightforward to implement, they are primarily suited for Single-Input-Single-Output (SISO) systems. To address this limitation, individual PID loops were designed for each joint, enabling control in this Multi-Input-Multi-Output (MIMO) system.

The PID controller was tuned using MATLAB's `slTuner`, which optimizes the gain values for each joint to achieve desired outputs. The tuned controller was tested with a polynomial trajectory input, where the arm was required to

follow a series of predefined waypoints over a set time interval. The simulation was conducted in MATLAB/Simulink, with the results evaluated based on the reference trajectory versus the actual joint positions.

The results show that while the PID controller tracks the reference trajectory, it struggles with accuracy and stability, especially for complex movements. These limitations highlight the need for more advanced controllers.

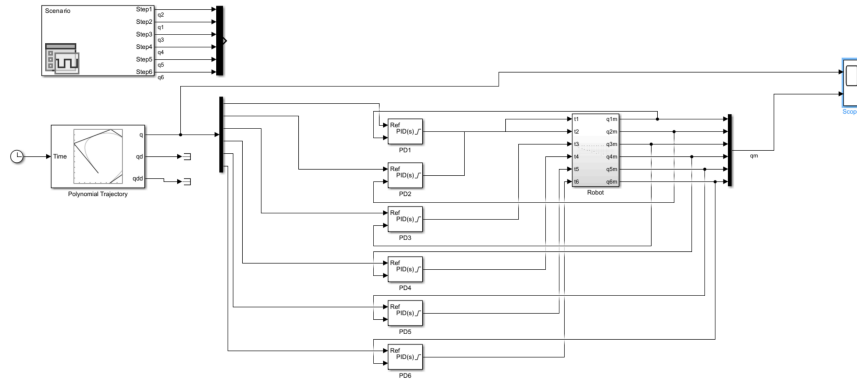
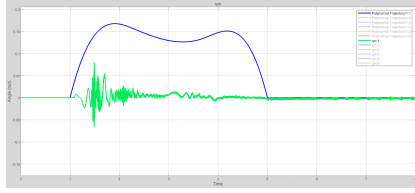
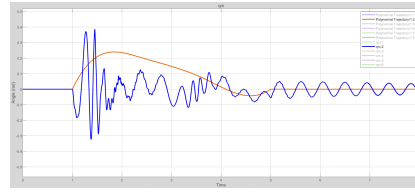


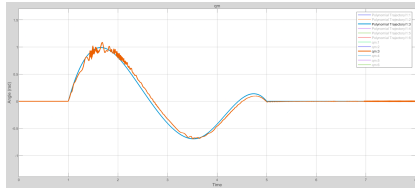
Figure 3: PID Simulink Model: The Simulink model showcasing the structure of the PID controller implemented for the robotic arm joints.



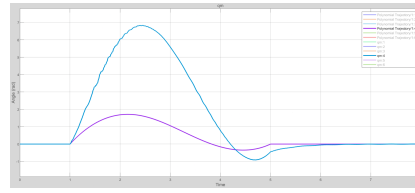
(a) PID Joint 1 Actual Position vs Reference Position



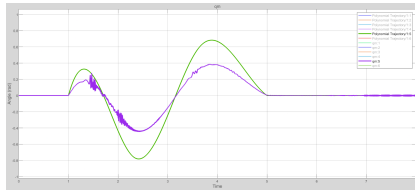
(b) PID Joint 2 Actual Position vs Reference Position



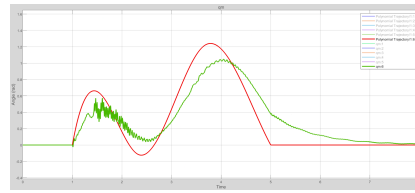
(c) PID Joint 3 Actual Position vs Reference Position



(d) PID Joint 4 Actual Position vs Reference Position



(e) PID Joint 5 Actual Position vs Reference Position



(f) PID Joint 6 Actual Position vs Reference Position

Figure 4: Simulation results showing actual positions vs reference positions for PID controllers across all six joints. Each plot corresponds to one of the six joints of the robotic arm.

5.2 LQR-Based Trajectory Tracking

In this project, the motion of a 6-axis robotic arm was controlled using a Linear Quadratic Regulator (LQR) to ensure precise trajectory tracking. The approach involved defining a set of waypoints for the joint angles, designing the LQR controller, simulating the system dynamics, and visualizing the results.

5.2.1 Defining Waypoints and Trajectories

The joint waypoints were defined as follows:

$$\text{Waypoints: } \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0.2 & 0.3 & 1.5 & 1.4 & 1 & 1.5 \\ 0.15 & 0.2 & 0.5 & 2.3 & -2.2 & -1.5 \\ 0.1 & 0.1 & -1.5 & -0.1 & 1.6 & 2.3 \\ 0.2 & -0.1 & 0.5 & -0.6 & 0.2 & 0.5 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

These waypoints represent the desired joint angles at specific time intervals. To generate smooth trajectories, a cubic spline interpolation was applied to the waypoints. The time span for the simulation was defined as 5 seconds, and the interpolated trajectory was sampled at 100 time steps.

5.2.2 LQR Control Design

The system dynamics were modeled using state-space representation with identity matrices for the state transition matrix A and the control input matrix B . The LQR controller was designed to minimize a quadratic cost function, balancing trajectory tracking accuracy and control effort. The cost matrices were defined as follows:

$$Q = \text{diag}(10, 10, 10, 10, 10, 10), \quad R = \text{diag}(1, 1, 1, 1, 1, 1),$$

where Q penalizes deviations from the desired state and R penalizes the control effort. The optimal feedback gain matrix K was computed using MATLAB's `dlqr` function.

5.2.3 Simulation and State Updates

The robotic arm's motion was simulated starting with all joints at zero angles. At each time step, the following steps were performed:

1. The desired trajectory at the current time step was computed using the interpolated spline.
2. The control input was calculated using the LQR control law:

$$u = -K \cdot (x - \text{desired_trajectory}),$$

where x is the current state vector.

3. The state was updated using Euler integration:

$$x_{\text{next}} = A \cdot x + B \cdot u.$$

This iterative process ensured that the actual joint angles closely followed the desired trajectories.

The LQR controller demonstrated effective trajectory tracking for the 6-axis robotic arm. The chosen waypoints resulted in smooth trajectories, and the control inputs remained within reasonable limits. This simulation validates the feasibility of using LQR for multi-axis robotic control, with potential applications in real-time hardware implementation.

5.2.4 LQR-Based Trajectory Tracking Results

The results of the LQR-based trajectory tracking are visualized in the figures below. These plots illustrate the effectiveness of the controller in following the desired joint trajectories and maintaining control input constraints.

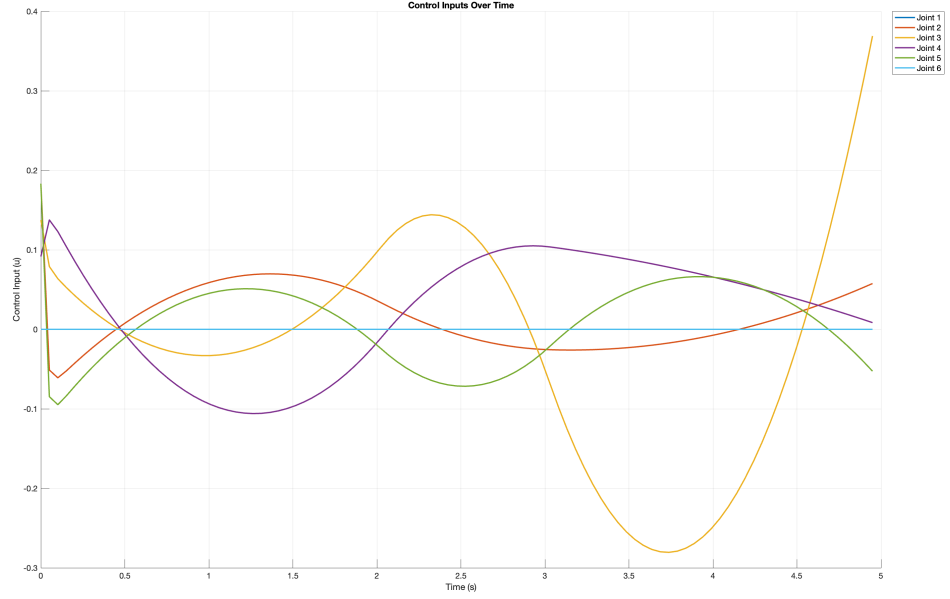


Figure 5: Control Inputs over time.

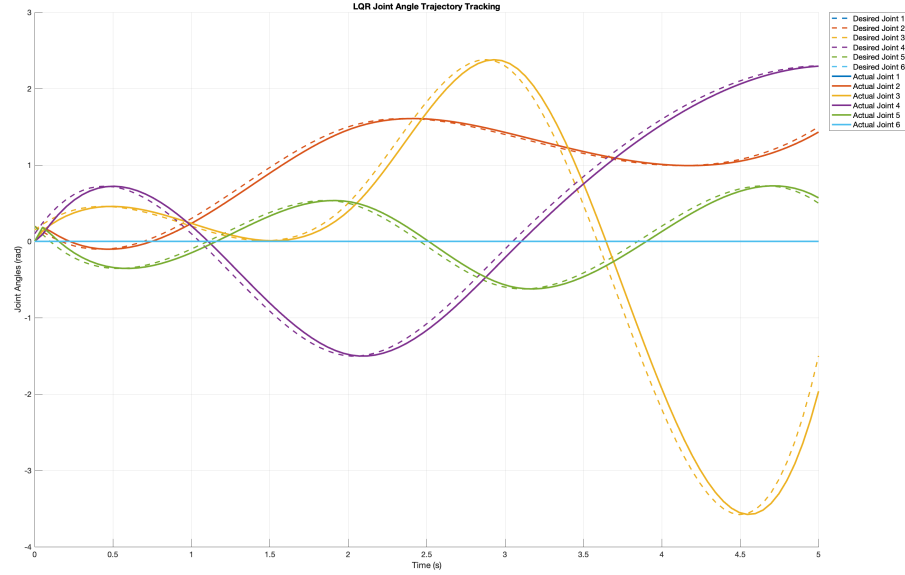


Figure 6: LQR joint angle trajectory tracking

6 Next Steps

6.1 Computed Torque Controller (CTC)

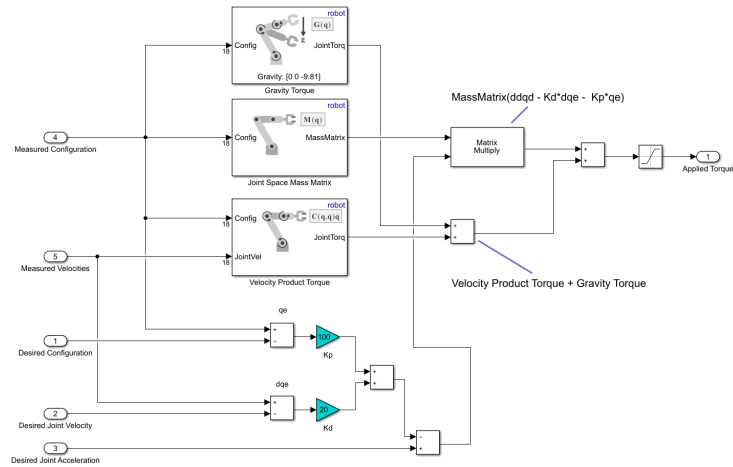


Figure 7: Computed Torque Controller in Simulink

The Computed Torque Controller (CTC) was explored as an advanced control framework for achieving precise trajectory tracking in robotic systems. Unlike PID, which is simpler and suitable for single-input-single-output (SISO) systems, CTC takes a more model-based approach. It leverages the dynamics of the robotic system, including the first and second derivatives of the reference position, joint positions, and velocities, to compute the required torques at each joint. The fundamental equation governing CTC is as follows:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau \quad (1)$$

Here:

- $M(q)$ is the inertia matrix,
- q represents the generalized coordinates,
- $C(q, \dot{q})\dot{q}$ is the Coriolis/Centripetal vector,
- $G(q)$ is the gravity vector, and
- τ is the computed torque.

The CTC approach was implemented in MATLAB/Simulink and used the same polynomial trajectory inputs as the PID controller for consistency in evaluation. While the gains for the controller were initially selected arbitrarily, this setup demonstrated promising results in tracking the desired trajectory compared to PID. However, the gains could be further tuned to ensure the controller operates within practical limits, such as torque constraints for real-world systems.

6.1.1 Advantages of CTC

Our simulations revealed that CTC significantly outperforms PID in trajectory tracking accuracy. The framework inherently compensates for system dynamics, resulting in smoother and more precise tracking. For example, as shown in the simulation results, the CTC closely follows the reference trajectory with minimal deviation, outperforming the PID controller, particularly for complex trajectories involving multi-input-multi-output (MIMO) systems.

6.1.2 Limitations and Areas for Improvement

While CTC has clear advantages, its dependency on an accurate model of the robotic arm introduces challenges:

- **Model Accuracy:** The performance of CTC heavily relies on accurate modeling of the robot's dynamics. Any errors in the inertia matrix, Coriolis terms, or gravity components can lead to suboptimal performance.
- **Computational Overhead:** The real-time computation of torque requires more resources compared to simpler controllers like PID. This can limit its use in systems with limited computational capabilities.

- **Parameter Tuning:** Although CTC required less manual tuning in our tests, further refinement of the gains would ensure better compliance with constraints like maximum allowable torque or limited acceleration, especially in sensitive applications.

7 Challenges, Learnings, and Future Directions

7.1 Challenges Encountered and Solutions

During the course of this project, several challenges were encountered:

- **Complexity in Tuning Controllers:** Designing and tuning controllers like PID and LQR for a 6-axis robotic arm posed significant difficulties due to the coupling between joints. This was addressed through careful parameter adjustments and iterative simulations to fine-tune the gains.
- **Model Accuracy:** The dynamic model of the robotic arm required accurate representation to ensure reliable simulation and controller performance. Discrepancies between the model and simulated results were mitigated by refining the parameters and incorporating validation tests.

7.2 Lessons Learned

The implementation of this project provided valuable insights:

- **Controller Strengths and Limitations:** Each control strategy, including PID, LQR, and CTC, has distinct strengths and limitations. While PID is simpler to implement, it struggles with multi-axis coupling, whereas LQR provides optimal control but requires accurate state-space modeling.
- **Importance of Model Validation:** Accurate modeling of the robotic arm's dynamics is critical for the success of any controller. Even small errors in modeling can lead to significant performance degradation in trajectory tracking.
- **Iterative Development:** Controller design and tuning require an iterative approach to balance performance, stability, and computational efficiency.

7.3 Next Steps

Future work on this project could involve:

- **Incorporating Data-Driven Approaches:** While traditional controllers like CTC and LQR perform well under known dynamics, integrating machine learning models can improve adaptability and performance in dynamic or uncertain environments.

- **Adaptive Control Strategies:** Exploring adaptive or hybrid control techniques to combine the robustness of traditional methods with the flexibility of data-driven models.
- **Enhanced Motion Planning:** Implementing advanced motion planning algorithms, such as RRT or machine learning-based planners, to improve trajectory optimization and collision avoidance in complex environments.

The rapid advancements in machine learning and control theory present exciting opportunities to blend traditional model-based design with data-driven methods, paving the way for more robust and efficient control strategies in robotics.

Appendix: LQR Control for a 6-Axis Robot Arm

The following MATLAB code implements the LQR control strategy for a 6-axis robotic arm, demonstrating trajectory tracking and control input calculation.

```

1 % Define waypoints for joint trajectories
2 wp = [0, 0, 0, 0, 0, 0;
3       0.2, 0.3, 1.5, 1.4, 1, 1.5;
4       0.15, 0.2, 0.5, 2.3, -2.2, -1.5;
5       0.1, 0.1, -1.5, -0.1, 1.6, 2.3;
6       0.2, -0.1, 0.5, -0.6, 0.2, 0.5;
7       0, 0, 0, 0, 0, 0];
8
9 % Define time span for waypoints
10 time = [0, 1, 2, 3, 4, 5];
11
12 % Interpolate waypoints to generate smooth trajectories
13 tspan = linspace(0, 5, 100); % Time steps for simulation
14 trajectory = interp1(time, wp', tspan, 'spline')';
15
16 % Define state-space matrices
17 A = eye(6); % State transition matrix
18 B = eye(6); % Control input matrix
19
20 % Define LQR cost matrices
21 Q = diag([10, 10, 10, 10, 10, 10]); % Penalizes state deviation
22 R = diag([1, 1, 1, 1, 1, 1]); % Penalizes control effort
23
24 % Compute LQR gain matrix
25 [K, ~, ~] = dlqr(A, B, Q, R);
26
27 % Initial state (all joints start at zero position)
28 x0 = zeros(6, 1);
29
30 % Simulate the system
31 x = zeros(length(tspan), 6); % Pre-allocate for state history
32 x(1, :) = x0'; % Initial state
33 dt = tspan(2) - tspan(1); % Time step size
34 u_history = zeros(length(tspan) - 1, 6); % Store control inputs
35
36 for i = 1:length(tspan) - 1

```

```

37     % Compute desired trajectory at time step
38     desired_trajectory = interp1(tspan, trajectory', tspan(i), '
        spline')';
39
40     % Calculate control input using LQR control law
41     u = -K * (x(i, :) - desired_trajectory);
42
43     % Update the state using Euler integration
44     x_next = A * x(i, :) + B * u;
45     x(i + 1, :) = x_next';
46
47     % Store control input
48     u_history(i, :) = u';
49 end
50
51 % Plot results
52 figure;
53 hold on;
54
55 % Plot desired trajectories (dashed lines)
56 colors = lines(6); % Color scheme
57 for i = 1:6
58     plot(tspan, trajectory(i, :), '--', 'LineWidth', 2, 'Color',
        colors(i, :));
59 end
60
61 % Plot actual trajectories (solid lines)
62 for i = 1:6
63     plot(tspan, x(:, i), '-', 'LineWidth', 2.5, 'Color', colors(i,
        :));
64 end
65
66 % Enhance plot aesthetics
67 xlabel('Time (s)', 'FontSize', 14);
68 ylabel('Joint Angles (rad)', 'FontSize', 14);
69 title('LQR Joint Angle Trajectory Tracking', 'FontSize', 16);
70 legend({'Desired Joint 1', 'Desired Joint 2', 'Desired Joint 3', '
        Desired Joint 4', 'Desired Joint 5', 'Desired Joint 6', ...
        'Actual Joint 1', 'Actual Joint 2', 'Actual Joint 3', '
        Actual Joint 4', 'Actual Joint 5', 'Actual Joint 6'},
        ...
        'FontSize', 12, 'Location', 'northeastoutside');
72
73 grid on;
74 set(gca, 'FontSize', 12);
75 hold off;
76
77 % Control input visualization (Optional)
78 figure;
79 hold on;
80 for i = 1:6
81     plot(tspan(1:end-1), u_history(:, i), 'LineWidth', 2, 'Color',
        colors(i, :));
82 end
83 xlabel('Time (s)', 'FontSize', 14);
84 ylabel('Control Input (u)', 'FontSize', 14);
85 title('Control Inputs Over Time', 'FontSize', 16);
86 legend({'Joint 1', 'Joint 2', 'Joint 3', 'Joint 4', 'Joint 5', '

```

```

87     Joint 6'}, ...
88     'FontSize', 12, 'Location', 'northeastoutside');
89 grid on;
90 set(gca, 'FontSize', 12);
hold off;

```

Listing 1: LQR Trajectory Tracking for a 6-Axis Robot Arm

8 Contributions

The contributions to this project were divided as follows:

- **Ronak Bhanushali:** Led the design and implementation of the PID and CTC control strategy for the 6-DOF robotic arm.
- **Abhijit Kaluri:** Worked on the implementation of the PID and CTC controllers. Contributed to the tuning and validation of the control parameters. Additionally, worked on plotting and generating results for PID
- **Kaustabh Chaudhari:** Focused on the design of the robotic arm simulation environment and setup. Contributed to the LQR control design and plots.
- **Harshil Mistry:** Led the design and implementation of LQR controller in MATLAB. Responsible for analysis and result verification as well as supervising the documentation process i.e. report formatting.

9 References

References

- [1] MathWorks, *Model and Control a Manipulator Arm with Simscape*, Accessed: 2024-12-02. <https://www.mathworks.com/help/robotics/ug/model-and-control-a-manipulator-arm-with-simscape.html>
- [2] MathWorks, *6DoF Robot Simulink Diagram for Simulation*, Accessed: 2024-12-02. <https://www.mathworks.com/matlabcentral/fileexchange/54455-6dof-robot-simulink-diagram-for-simulation>
- [3] MathWorks, *Multi-Loop PID Control of a Robot Arm*, Accessed: 2024-12-02. <https://www.mathworks.com/help/control/ug/multi-loop-pid-control-of-a-robot-arm.html>